📘 **AI/ML Internship – Day 11 Notes & Tasks**

🟢 **Module 3: Python for AI**

---

📌 **Part 1: Inheritance**

🔹 **What is Inheritance?**

Inheritance allows a class to **reuse properties and methods** from another class.

👉 Parent Class → Base class
👉 Child Class → Derived class

---

🔹 **Why Use Inheritance?**

- Code reuse
- Less duplication
- Easy maintenance

---

🔹 **Example**

```
class Person:
    def __init__(self, name):
        self.name = name

    def display(self):
        print("Name:", self.name)

class Student(Person):
    def __init__(self, name, marks):
        super().__init__(name)
        self.marks = marks

s1 = Student("John", 85)
s1.display()
print(s1.marks)
```

---

📌 **Part 2: Method Overriding**

### 🔹 What is Method Overriding?

Child class can **modify parent class method**.

---

```
class Person:
    def display(self):
        print("This is Person")

class Student(Person):
    def display(self):
        print("This is Student")

s = Student()
s.display()
```

---

## 📌 Part 3: Encapsulation

### 🔹 What is Encapsulation?

Encapsulation means **hiding internal data** and controlling access.

---

### 🔹 Private Variables

```
class Student:
    def __init__(self, name, marks):
        self.name = name
        self.__marks = marks   # private variable

    def get_marks(self):
        return self.__marks

s = Student("Sara", 90)
print(s.get_marks())
```

---

### 🔹 Why Encapsulation?

- Data protection
- Controlled access

- Prevent misuse

---

### 📌 Part 4: Real-World Example

```
class Employee:
    def __init__(self, name, salary):
        self.name = name
        self.__salary = salary

    def get_salary(self):
        return self.__salary

class Manager(Employee):
    def __init__(self, name, salary, department):
        super().__init__(name, salary)
        self.department = department

m = Manager("Alex", 50000, "IT")
print(m.get_salary(), m.department)
```

---

### 📌 Part 5: Types of Inheritance

#### 🔹 Single Inheritance

One parent → one child

#### 🔹 Multiple Inheritance (basic idea)

One child → multiple parents

```
class A:
    pass

class B:
    pass

class C(A, B):
    pass
```

---

### 📌 Part 6: Real-World Thinking

👉 In AI/ML:

- Models = classes
- Different models = inheritance
- Data protection = encapsulation

👉 Example:

- Base model → parent
- Custom model → child

---

🎯 **Tasks for Day 11**

---

🧠 **Task 1: Theory**

1. What is inheritance?
2. What is method overriding?
3. What is encapsulation?
4. What is private variable?
5. Why OOP is used in real projects?

---

💻 **Task 2: Inheritance Practice**

1. Create class Animal
2. Create class Dog inheriting Animal
3. Add method sound()

---

💻 **Task 3: Method Overriding**

1. Parent class → Vehicle
2. Child class → Car
3. Override method start()

---

### 💻 Task 4: Encapsulation Practice

1. Create class BankAccount
2. Make balance private
3. Create methods:
    - deposit
    - withdraw
    - check balance

---

### 💻 Task 5: Combined Practice

👉 Create:

- Class Person
- Class Employee (inherits Person)
- Add:
    - name
    - salary
    - display details